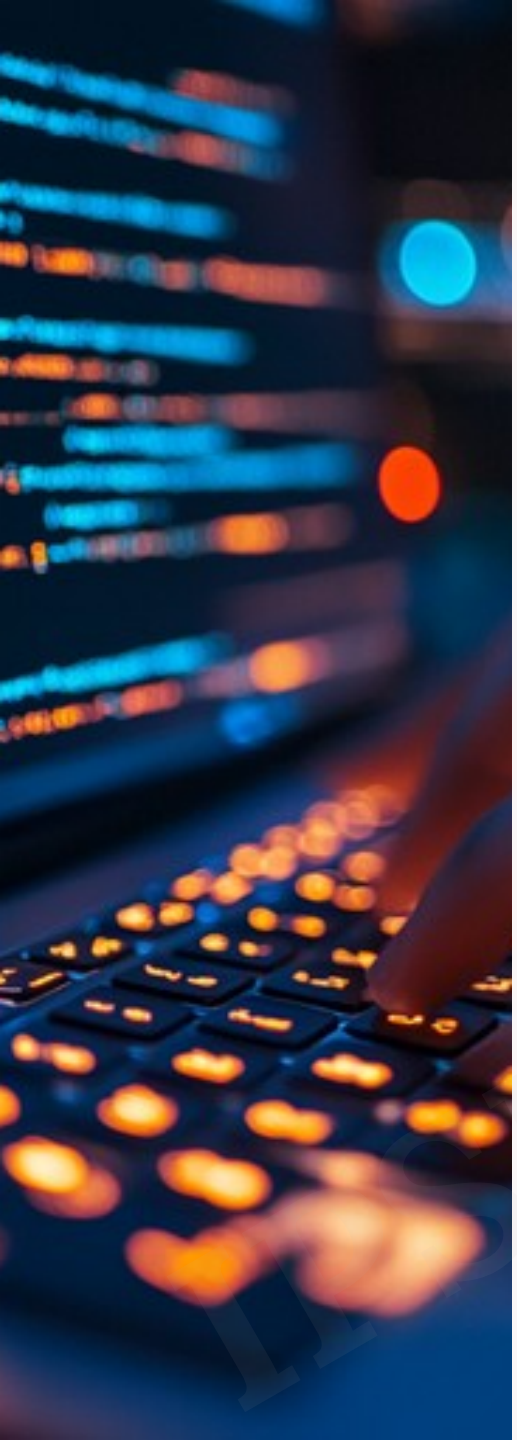


第五章、實際案例：Arm 處理器 - 02

李建緯
雲科大資工系 智慧感知與系統整合實驗室
IPSI Lab, Yuntech-CSIE

IPSI Lab, Yuntech-CSIE

Arm 組合語言程式架構

程式架構

Arm 組合語言原始檔（`.s`）需要分清楚兩個不同層次：

- **原始碼元素**：Directive（組譯器指示）、Label / Symbol（標籤 / 符號）、Instruction（機器指令）。
- **程式區段（Section）**：`.text` 放指令，`.data`、`.rodata`、`.bss` 放資料或保留空間。

本單元依序介紹三類常見內容：

1. **組譯器指示與符號**：`.global`、`.equ`、`_start:`。
2. **程式碼區段**：`.text` 與組合語言指令。
3. **資料區段**：`.data`、`.rodata`、`.bss`。

注意： 上述是常見的原始碼組織方式，**不是 ELF 規定的三個獨立區段**。`.text`、`.data` 同時是 Section 名稱及切換區段的 Directive。

本單元環境： GNU Assembler / AArch32（A32），銜接後續 MSYS2、`arm-none-eabi`、QEMU `versatilepb` 實作。

一份 `.s` 裡的三種不同內容

```
.global _start              @ Directive：告知組譯器/連結器符號資訊
.data                       @ Directive：切換資料區段
num1: .word 10              @ Label + Directive：定義資料及其名稱
.text                       @ Directive：切換程式碼區段
_start:                     @ Label：為後續指令指定位置
    MOV R0, #10             @ Instruction：由 CPU 執行
```

類型	對誰產生作用？	在 ELF 中的表現
Directive	Assembler / Linker	建立區段、符號、資料
Label	Assembler / Linker	對應位置或符號名稱
Instruction	CPU	編成機器碼，執行時改變狀態

閱讀重點： Directive / Label 是提供組譯及連結所需資訊；Instruction 才會被編成供 CPU 執行的機器碼。下一節再分別說明設定、資料與程式碼。

1. 組譯器指示與符號設定（Directives & Symbols）

這些內容是組合語言原始碼中的設定與名稱，而非正式的「Header Section」。

- `.text` 、 `.data` ：切換目前輸出至哪個 Section。
- `.global _start` ：讓 `_start` 符號可供連結器辨識。
- `_start:` ：為所在位置建立標籤，並非 CPU 自動認識的指令。

> **重點：** `.global _start` **不會直接設定 ELF Entry Point。連結器負責依設定決定入口位址；Bare-metal 還須配合目標平台的載入與啟動流程。**

- **常見指令**：
  - `.global <label>`：宣告全域符號，使連結器可看見它；**入口點仍由連結器設定**（常透過 `_start` 慣例或連結選項）。
  - `.equ <name>, <value>`：在組譯時定義符號常數，常用於硬體暫存器位址或固定參數。
  - `.extern <symbol>`：可宣告由其他檔案提供的外部符號；GNU Assembler 即使沒有此宣告，也可對未定義符號建立外部參照。

範例：Directive、Label 與 Instruction

```
.global _start      @ Directive：宣告全域符號
.text               @ Directive：切換至程式碼區段
_start:             @ Label：定義這個位置的名稱
    MOV R0, #0      @ Instruction：CPU 執行的指令
    B .             @ 示意：停在目前位址
```

注意： **上述範例仍需符合目標板的連結、載入與啟動設定；此處重點是識別原始碼元素。**

`.global _start` 不等於 CPU 立刻從這裡執行

```
.global _start          @ 宣告符號可被連結器看見
.text
_start:                 @ 把名稱綁定到這個位址
    B _start
```

同一段程式會經過三個不同層次：

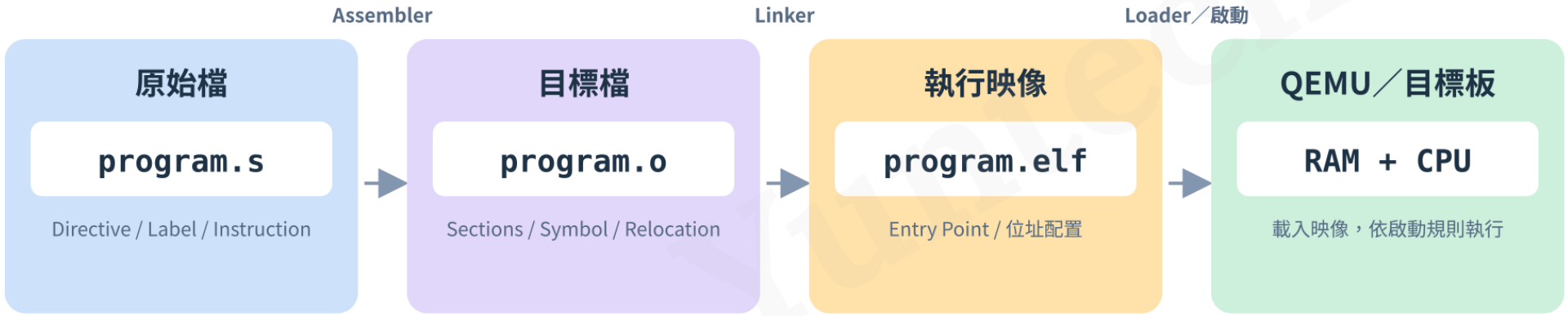
層次	工作
Assembler	為 `_start` 建立符號，將指令轉成目標檔
Linker	依連結設定決定段落放置、ELF Entry Point
Loader / 啟動流程	將可執行映像載入目標記憶體並開始執行

後續 Toolchain 單元可用 `readelf -h` 檢查 Entry point address、`readelf -s` 檢查 Symbol Table。兩者不應混為一談。

從原始檔到執行：各階段如何接力？

從 .s 到執行：Assembler、Linker、Loader 分工

符號、區段和程式入口是在不同階段處理；Directive 不等於 CPU 指令



`.global _start` = 宣告符號可見，不是直接設定 ELF Entry Point

Entry Point 由連結設定決定；實際載入位置與啟動方式仍須符合目標平台。

CPU-02：學懂原始檔角色

後續工具：objdump、readelf、GDB（GNU Debugger）核對符號、位址與機器指令

先釐清： `.global` **與 Label 建立符號；ELF Entry Point 由連結設定決定，真正開始執行還要經過目標板的載入 / 啟動流程。**

2. 資料段（Data Section）

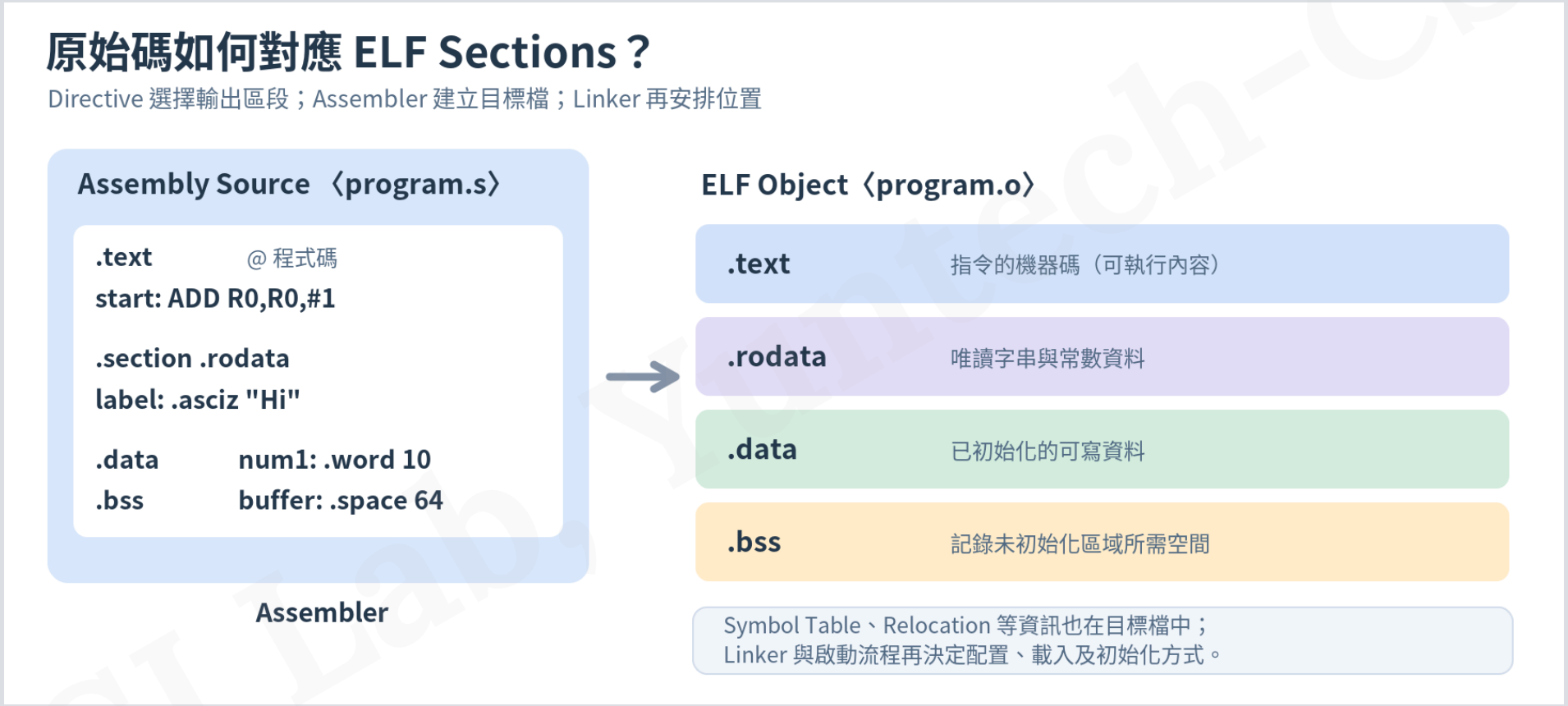
- **主要功能**：
  - 定義常數、變數等在程式執行期間可能需要的資料。
  - 使用標準指令 `.data` 或 `.section .data` 開始。
- **常見指令**：
  - `.data`：切換到可寫、**已初始化資料**區段；`.rodata` 常用於唯讀常數，`.bss` 用於未初始化資料。最終位址與初始化方式由 Linker Script / 啟動流程決定。
  - `.word <value>`：定義一個 32 位元的整數資料。
  - `.byte <value>`：定義一個 8 位元的整數資料。
  - `.ascii "<string>"`：定義一個 ASCII（American Standard Code for Information Interchange）字串，不會自動在末尾添加 Null 終止符。
  - `.asciz "<string>"`：定義一個 ASCII 字串，並自動在末尾添加 Null 終止符 (C 語言風格字串)。

範例

```
.data
num1: .word 10          @ 定義一個 32-bit 整數
char1: .byte 65         @ ASCII 'A' = 65，定義一個 byte
msg: .asciz "Hello, Arm!\n" @ C-style 字串
```

IPSI Lab, Yuntech-CSIE

資料與程式碼如何形成 ELF Sections？



閱讀重點： Directive 決定程式碼或資料進入哪個 Section；Assembler 產生 `.o`，Linker 再安排位址。`.bss` 通常只記錄所需空間；Bare-metal 的啟動流程須確保正確初始化。圖中不是固定實體位址。

3. 程式段（Code Section）

- **主要功能**：
  - 包含程式的核心指令，完成資料的運算、流程控制、函式呼叫等操作。
  - 使用標準指令 `.text` 或 `.section .text` 開始。
- **常見指令**：
  - `.text`：切換至程式碼區段；最終可能配置到 ROM（Read-Only Memory）、Flash 或可執行 RAM（Random Access Memory），由連結設定及目標平台決定。
  - `_start:`：常見的程式入口符號名稱，**是否作為執行入口**仍須由連結與啟動設定決定。
  - `function_name:`，函數標籤。用於定義子函數的入口。

- **程式流程**：
  - 設定暫存器並載入初始資料
  - 執行邏輯或算術運算
  - 進行條件分支或迴圈操作
  - 程式如何結束取決於執行環境；本課程的 Bare-metal/QEMU 範例通常以迴圈停留，並非直接「回到作業系統」。

範例

```
_start:
    LDR R0, =num1          @ GNU Assembler 偽指令：取得 num1 位址
    LDR R1, [R0]           @ 由 num1 位址讀取內容
    ADD R1, R1, #5         @ 先只修改 R1
    STR R1, [R0]           @ 寫回記憶體中的 num1
    B .                    @ Bare-metal 示意：停在此處
```

IPSI Lab yuntech-CSIE

常見標籤（Labels）與用途

以下標籤為程式設計者常用的命名習慣，而非語言保留字：

- `_start`：程式進入點標籤，通常在 `.global` 指令中指定。
- `main`：一般用於子程式或主要邏輯段落，便於程式結構化。
- `loop`：定義迴圈結構的起始點。
- `end`：程式結尾區域。不具語意，不會自動結束程式。

IPSI Lab, Yuntech-CSIE

完整範例程式碼

```
.syntax unified
.global _start          @ 宣告全域標籤
.data                   @ 資料段開始
.align 2                @ GNU as：下一個位置對齊到 2^2 = 4 bytes
num1: .word 10          @ 變數 num1

.text                   @ 程式段開始
_start:
    LDR R0, =num1       @ 載入變數位址
    LDR R1, [R0]        @ 讀取 num1
    ADD R1, R1, #5      @ R1 = R1 + 5
    STR R1, [R0]        @ 寫回數值
stop:
    B stop              @ Bare-metal：停留在此，不會自動返回作業系統
```

IPSI Lab, Yuntech-CSIE

追蹤完整範例：`num1` 為什麼變成 15？

延續上一張程式（初始 `num1 = 10`），依 AArch32 指令語意觀察：

執行步驟	R0	R1	記憶體 `num1`
起始	未指定	未指定	10
`LDR R0, =num1`	`&num1`	不變	10
`LDR R1, [R0]`	`&num1`	10	10
`ADD R1, R1, #5`	`&num1`	15	10
`STR R1, [R0]`	`&num1`	15	**15**

從原始碼到觀察： `.word 10` 初始化資料 → LDR 取得數值 → ADD 只改 R1 → STR 才真正改變 `num1` 。

後續使用 GDB（GNU Debugger）可透過 `info registers r0 r1` 和 `x/wd <位址>` 驗證這個過程。

將 num1 的變化畫出來：Register vs Memory

同一份程式：暫存器改變，不代表記憶體立即改變

初始 num1 = 10，R0 = &num1（符號位址，不假設實際 RAM 地址）

① Load

```
LDR R1, [R0]
```

暫存器	記憶體
R1 = 10	num1 = 10

LDR 只讀取，不修改 num1

② Compute

```
ADD R1, R1, #5
```

暫存器	記憶體
R1 = 15	num1 = 10

R1 改變，num1 保持不變

③ Store

```
STR R1, [R0]
```

暫存器	記憶體
R1 = 15	num1 = 15

到 STR 才真正更新 num1

重點：LDR 是記憶體 → 暫存器；ADD 是暫存器內運算；STR 才將結果寫回記憶體。

可在後續 GDB 分別觀察 R0、R1 及 &num1 指向的記憶體數值。

要點： `LDR` 只把資料載入 R1，`ADD` 只改暫存器；到 `STR` 才把 15 寫回 `num1`。

C語言與組合語言混用

C語言嵌入或呼叫組合語言

- **控制特定指令 / 硬體**：在 C 程式中嵌入組合語言，可存取部分特殊指令；**不保證比編譯器產生的程式碼更快**。
- **處理特殊運算**：在需要特殊指令或精確控制的情境可能有助益；一般運算未必優於編譯器最佳化結果。
- **嵌入式系統應用**：必要時可直接使用特殊硬體指令；一般 MMIO（Memory-Mapped Input/Output，記憶體對映輸入 / 輸出）也可透過 C 的 `volatile` 存取與驅動程式處理，**不一定需要手寫組合語言**。

IPSI Lab, Yuntech-CSIE

使用 `__asm__` 嵌入組合語言

GCC（GNU Compiler Collection）提供 `__asm__` 或 `asm`，在 C 程式中插入目標架構的組合語言；本單元採 AArch32 / A32。

- **Basic Asm**：`__asm__("nop");`，不指定 C 變數的輸入與輸出。
- **Extended Asm**：透過下列欄位告訴 GCC 如何傳遞操作數及處理副作用。

```
__asm__ ("assembly instructions"
         : output_operands    /* 輸出 */
         : input_operands     /* 輸入 */
         : clobbers           /* 其他副作用 */);
```

注意：上面是語法結構示意，非可直接編譯的完整程式；後頁的加法範例使用實際 Constraint。GCC 會據此配置暫存器並進行最佳化。

範例：簡單的加法運算

```c
#include <stdio.h>

int add_numbers(int a, int b) {
    int result;
    __asm__ (
        "ADD %[res], %[val1], %[val2]\n"
        : [res] "=r" (result)
        : [val1] "r" (a), [val2] "r" (b)
    );
    return result;
}

int main() {
    int sum = add_numbers(5, 10);
    printf("5 + 10 = %d\n", sum);
    return 0;
}
```

IPSI Lab, Yuntech-CSIE

說明：

- `ADD %[res], %[val1], %[val2]`：使用 `ADD` 指令將 `val1` 和 `val2` 加起來，並儲存到 `res`。
- `: [res] "=r" (result)`：`=` 是**唯寫輸出**，`r` 讓 GCC 配置適合的通用暫存器。
- `: [val1] "r" (a), [val2] "r" (b)`：設定輸入操作數；`r` 代表**由編譯器選擇適合的通用暫存器**，並非指定某個固定暫存器。

IPSI Lab, Yuntech-CSIE

GCC Extended Asm：從 C 變數到機器指令

Inline Assembly：指令如何與 C 變數連接？

GCC Extended Asm：Template : Outputs : Inputs : Clobbers

```
__asm__ ( "ADD %[res], %[val1], %[val2]"
          : [res] "=r" (result)          : [val1] "r" (a), [val2] "r" (b) );
```



Clobbers／副作用： ADD（不帶 S）不更新條件旗標；若改為 ADDS，需宣告 "cc"。

"r" 表示允許編譯器配置通用暫存器，並不是強制使用 R0、R1。

閱讀重點： `a=5` 、 `b=10` 是兩個輸入；`"r"` 允許 GCC 配置適合的暫存器；A32 `ADD` 運算後，由 `"=r"` 將結果交回 C 變數 `result` 。 `r` **不代表固定使用 R0。**

使用外部組合語言函式

- **分離程式碼**：可以將組合語言寫在獨立的 `.s` 檔案中，並在 C 程式中呼叫該函式。
- **方便調試與維護**：將組合語言寫在單獨的檔案有助於調試與檔案管理。

IPSI Lab, Yuntech-CSIE

範例：外部組合語言加法函式

1. 加法函式（add.s）

> 註：此獨立函式遵循 AAPCS32 呼叫慣例；LR 為連結暫存器。

```armasm
.syntax unified
.text
.global add_numbers
.type add_numbers, %function
add_numbers:
    ADD R0, R0, R1          @ AAPCS32：R0、R1 傳入，結果保留 R0
    BX LR                   @ Leaf function：返回 Caller
.size add_numbers, .-add_numbers   @ ELF symbol 的函式大小
```

IPSI Lab, Yuntech-CSIE

2. C 程式碼（main.c）

```c
#include <stdio.h>

extern int add_numbers(int a, int b);

int main() {
    int sum = add_numbers(5, 10);
    printf("5 + 10 = %d\n", sum);
    return 0;
}
```

IPSI Lab, Yuntech-CSIE

3. 編譯與連結

```
arm-none-eabi-gcc -mcpu=arm926ej-s -marm \
  -o program.elf main.c add.s syscalls.c \
  -specs=nosys.specs -lc -lgcc
```

採用後續 MSYS2 / `arm-none-eabi` / QEMU Bare-metal 實驗環境；`-mcpu=arm926ej-s -marm` 明定 Armv5TE / A32 目標。`syscalls.c` 暫沿用**現有已可運作版本**，不在本次修改。

說明：

- 在 `add.s` 檔案中，`add_numbers` 函式使用 `ADD` 指令執行加法，並返回至呼叫端。
- 在 `main.c` 中，使用 `extern` 宣告該組合語言函式並進行呼叫；由 AAPCS32 保證跨語言的參數與返回值約定。

C 呼叫外部 Assembly：參數與返回值走哪裡？



回扣 CPU-01： `BL` 設定 LR；`ADD` 由 R0/R1 取得參數、結果留在 R0；`BX LR` 返回。`add.s` 不呼叫其他函式，是不需保存 LR 的 Leaf Function。

下一步觀察： 後續利用 `objdump`、GDB 檢查 `BL`、`BX LR` 及 R0 的變化。

使用內聯組合語言的技巧

前一單元的外部 `.s` 函式透過 AAPCS32 與 C 協作；回到 Inline Assembly 時，程式碼直接嵌在 C 函式內，由 GCC 的 Operand Constraints 與 Clobbers 協調暫存器和副作用。

- **輸入 / 輸出**：`"r"` 允許編譯器分配一般用途暫存器，不是指定 R0。
- **讀寫限制**：`"=r"` 、 `"+r"` 、 `"=&r"` 描述輸出、讀寫及提早覆寫的不同需求。
- **其他副作用**：用 `"cc"` 描述條件旗標變化；必要時用 `"memory"` 告知未列在操作數中的記憶體存取。

注意： **Inline Assembly 不能因為寫了 `asm` 就假定比 C 更快；必須讓 GCC 正確知道它可能影響哪些資料。**

Inline Assembly：Constraint 與 Clobber 不只是語法

標記	告訴 GCC 的事情	本單元應注意
`"r" (x)`	需要**可讀取**的通用暫存器輸入	不保證使用 R0
`"=r" (y)`	指令會寫入輸出暫存器	舊值不用保留
`"+r" (x)`	同一操作數既讀又寫	常用於原地更新
`"=&r" (y)`	唯寫輸出，且屬 **Early-clobber**	輸出若會在所有輸入讀完前就被寫入，需避免不安全的暫存器重疊
`"cc"` / `"memory"`	告知 GCC 會改變條件旗標 / 存取未列於操作數的記憶體	依實際指令副作用宣告；`"memory"` 不等於 CPU 硬體記憶體屏障

下一頁以會改變 NZCV（Negative、Zero、Carry、Overflow）條件旗標的 `ADDS`，示範 `"+r"` 與 `"cc"`；`"=&r"` 則用於確實具有提早覆寫風險的指令序列。

Inline Assembly：`+r` 與 `cc` 的完整例子

例如 AArch32（以 `unsigned` 示範位元運算）：

```
unsigned inc(unsigned value) {
    __asm__ ("adds %[v], %[v], #1"
             : [v] "+r" (value)
             :
             : "cc");   // ADDS 會更新條件旗標
    return value;
}
```

重點： 由 Compiler 配置暫存器；Constraint 不完整可能使最佳化後的程式行為錯誤。

IPSI Lab, Yuntech-CSIE

範例：計算數值乘以 2 的運算（LSL：Logical Shift Left）

```c
#include <stdio.h>

int multiply_by_two(int num) {
    int result;
    __asm__ (
        "LSL %[res], %[val], #1\n"
        : [res] "=r" (result)
        : [val] "r" (num)
    );
    return result;
}

int main() {
    int result = multiply_by_two(7);
    printf("7 * 2 = %d\n", result);
    return 0;
}
```

IPSI Lab, Yuntech-CSIE

延伸說明：LSL ×2 與 C 的整數型別

- 此例 `7` 左移 1 bit，得到 **14**；LSL 是位元層級操作。
- 不要因此推論對**任意 signed `int`** 都能無條件等價於 C 的 `num * 2`：有號溢位及負數的語言語意需分開討論。
- 若目的是觀察 32-bit 位元運算，可把範例參數改成 `unsigned int`；要討論 C 語言乘法，則應明確約束值域。
- **教學重點**：理解此例指令如何運作，不是證明手寫 Inline ASM 一定比 C 編譯器更有效率。

IPSI Lab, Yuntech-CSIE

外部 `.s` 與 Inline Assembly：何時選哪一個？

需求	外部 Assembly（`.s`）	Inline Assembly（`__asm__`）
完整函式、獨立測試	較合適	可行，但較不易獨立重用
少量特殊指令嵌入 C	需要額外呼叫介面	較方便
暫存器與參數處理	遵守 AAPCS32 呼叫慣例	由 GCC Constraints 與 Clobbers 協調
除錯	可反組譯並逐步追蹤	也可追蹤，但須理解編譯器生成的周邊指令

與 CPU-01 的連結： `R0/R1` 傳參數、`R0` 回傳、`BL/BX LR` 控制流程、`LDR/STR` 處理記憶體。

銜接實作： 下一單元使用既有 Windows / MSYS2、`arm-none-eabi-gcc`、QEMU `versatilepb` 與 GDB，觀察 Section、Symbol、反組譯與暫存器。`syscalls.c` 本次維持原有已可運作版本。